Mission 005: Canonical DB Architecture Mission Packet

This mission replaces the snapshot-first model with a shared database model: agents write to the backend through an API, and the application reads from the same database through read APIs.

User goal

Implement the new architecture so the data behind Agents-Vis has one canonical source of truth in the database, with agents writing through a backend API and the UI reading from the same data path.

Core product decision

Use a single canonical database as the source of truth. Do not keep JSON replication as the primary data path. Keep the app read-only for end users, but allow agents to write through a backend API.

Scope in

• One canonical database as the source of truth

• Agent write API for mission events and status updates

• Read API for dashboard and latest mission views

• Stable data contract shared by backend and frontend

• Freshness / lag / updated-at metadata on reads

• Migration or bootstrap path from the current live source into the database

• Read-only UI that reflects the DB-backed state

• Production-safe verification on the deployed app

• Clear separation between source of truth, write path, read path, and UI

Scope out

• End-user mutation controls

• Admin dashboard features

• A second dashboard view

• JSON replication as the primary architecture

• Webhook-only ingestion if a simpler push API or polling path is safer

Team roles and handoff

• Architect owns the system design, data model, API contracts, consistency rules, migration plan, and boundary decisions

• Product Manager owns UX/UI direction, narrative clarity, and what the user should see

• Backend Developer owns the database schema, write API, read API, and backend tests

• Frontend Developer owns the read-only UI and live-state rendering

• QA owns validation, regression checks, and end-to-end verification

• Coordinator owns sequencing, blockers, release gates, and status updates

• Every role must write a durable handoff after its scoped task

Architecture contract

• Agents never write directly to the browser app

• Agents write events to a backend API

• The backend validates and persists events in the canonical DB

• The app reads from read APIs that query the same DB

• Optional realtime delivery may be layered on later, but is not required for the core architecture

• If a JSON export is kept, it must be an artifact, not the source of truth

Architect decisions folded in

• Canonical production and preview storage is Neon via the user's Vercel setup.

• The smallest v1 architecture is append-only agent writes plus DB-backed read APIs.

• Recommended tables are `missions` for current canonical state and `mission_events` for immutable history.

• The write path should be a simple `POST /api/agent-events` that validates payload shape and writes to the canonical DB.

• Read APIs should continue to expose the existing dashboard and latest-mission response envelopes.

• Freshness should be derived from DB timestamps and surfaced as `fresh`, `partial`, `delayed`, `stale`, or `empty`.

• JSON-backed runtime loading should stop being the truth path after migration/backfill.

• Realtime push is later-phase only; polling/read APIs are sufficient for v1.

Acceptance criteria

• Agents can write mission events through a backend API

• The database persists the canonical mission state

• The dashboard API reads from that same database

• The UI reflects changes without relying on JSON replication as the main path

• The current read-only dashboard experience still works

• Freshness and lag metadata are visible and accurate

• Backend, frontend, and QA tests pass

• Production smoke verification confirms the live app is reading from the DB-backed path

Open questions

• Implementation details for Neon client wiring, migrations, and env var names.

• Exact UI copy for freshness labels and lag formatting.

• Whether export/debug artifacts should be generated on a schedule after cutover.

Dependencies / blockers

• Agreement on the agent write API contract

• Agreement on the canonical event schema and migration plan

• Production or preview verification access for smoke testing

• Neon is available through the user's Vercel setup, so database provisioning is not a blocker

Notes

• This mission intentionally treats the JSON file as an export or debug artifact, not the main architecture

• Keep the UI simple and read-only

• Prefer the smallest architecture that gives a single source of truth and a stable API

• Create and keep the mission documentation in PDF format alongside the markdown docs